Fundamental Project Report

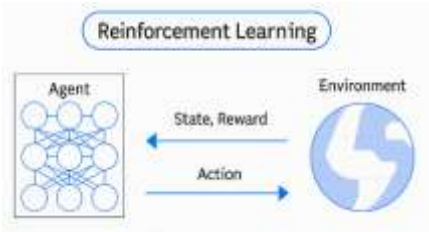
RL Project 2025

Soji John
0626734

June 10, 2025

1. INTRODUCTION

Reinforcement learning (RL) is a type of machine learning process where an agent learns to make decisions by interacting with its environment using a reward-and-punishment paradigm, to maximize cumulative reward. It is a computational approach to learning from interaction how to behave in order to achieve a goal.



In RL, the key components are:

1. Agent: This is the entity that observes the environment and takes actions.
2. Environment: The world with which the agent interacts. It defines the rules, states, and rewards.
3. State (s_t): A snapshot of the environment at a given time.
4. Action (a_t): A move made by the agent in the environment at a given time.
5. Reward (r_t): A scalar feedback signal given by the environment to the agent at time t after an action is taken. It indicates how good or bad the agent's action was.
6. Policy (π): A policy is a decision-making rule used by an agent to decide what actions to take.

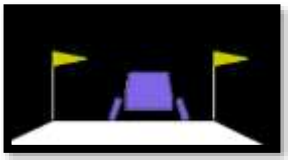
The goal of RL is to compute policy π that maximizes the expected cumulative return. **Policy optimization** refers to methods that optimizes the policy by directly **adjusting the parameters** to maximize expected return. There are two main types of methods:

- **Gradient-Based** methods (eg, Policy Gradient) compute the gradient of the expected return with respect to the policy parameters and use it to update the policy. Gradient-based methods are well-suited for complex high-dimensional environments where actions are continuous (e.g., controlling a robot arm). However, it can suffer from high variance in gradient estimates, leading to unstable training, and may also get stuck in local optima.
- **Gradient-Free** methods (eg, Zeroth Order, Population Method), rely on evaluating the function at different points and using that information to guide the search for an optimum. This is achieved by iteratively 'perturbing' the parameters and then evaluating the policy to maximize expected return. These methods are preferred when the policy space is large and has many local optima, handles noise better, and is often simpler to implement.

2. EXPERIMENT:

In this project, for the 'Lunar Lander' task, we employ two gradient-free optimization algorithms: **Zeroth-Order** optimization and **Population-based** optimization. In both these algorithms, we iteratively perturb the policy parameters and then evaluate the policy with the goal of finding the parameter configuration that maximises the expected cumulative return of the task.

2.1 Environment Setup:



For this experiment, we use the **LunarLanderContinuous-v3** environment provided by Gymnasium library (formerly OpenAI Gym), wherein an agent learns to land a lunar module safely on a landing pad using continuous thrust commands. The environment provides a scalar reward which serves as the optimization signal for the gradient-free learning method, and therefore is ideal for RL experiments.

State Space:	8-dimensional vector: Position (x, y) and velocity (vx, vy), angle θ and angular velocity ω, Leg contact binary indicators (l1, l2)
Action Space:	2D continuous vector: [main engine throttle, side engine throttle]
Reward Function:	Combines multiple factors: - Landing success (Reward for safe landing, Penalty for crashing) - Bonus for landing upright with both legs - Penalty for fuel usage (to encourage efficiency)
Goal:	Learn a policy that maximizes **total reward** by safely and efficiently landing the spacecraft

2.2 Zeroth-Order Optimization Algorithm:

Zeroth-order (ZO) optimization is a gradient-free approach used when direct gradient computation is impractical or impossible. Instead of relying on gradients, ZO methods use function evaluations to iteratively improve policies or parameters.

<u>Algorithm Steps performed:</u>

1. Implemented parametric policy network by using simple neural network with 8 inputs, 128 hidden neurons, and 2 outputs. Policy Optimization iteration starts with the below step.
2. Each optimization (episode) step begins by creating two cloned copies of the current policy: one for positive perturbation **θ+** (policy_pos) and another for the negative perturbation **θ−** (policy_neg)
3. Created Gaussian Perturbation ϵ (noise = torch.randn_like(param) * std_dev σ = 0.02), where ϵ is sampled from a zero-mean Gaussian distribution.
4. The ±Perturbation is applied to the two policies: Positive perturbation θ+ϵ (policy_pos) and negative perturbation θ−ϵ (policy_neg).
5. Evaluated both policies (policy_pos, policy_neg) and averaged over 5 Evaluations per episode for averaging to reduce noise in reward scores, to see which of the two perturbed policy performed better.
6. Calculated Gradient = (0.5 x (Score of positive perturbation - Score of negagive perturbation) x Perturbation).

$$\hat{\nabla}_\theta J(\theta) \approx \frac{f(\theta+\epsilon) - f(\theta-\epsilon)}{2} \cdot \epsilon$$

7. Updated the policy parameters (param.data += learning_rate x gradient) in the direction that improves performance (i.e., towards the better-scoring perturbation).

   $$\theta \leftarrow \theta + \alpha \cdot \hat{\nabla}_\theta J(\theta)$$

8. Maximum of the two score from both perturbations is logged as the best score for this given episode.
9. Repeated steps 2-8 for 5000 Episodes (training iterations) each involving perturbation and evaluation.
10. Learning curve plotted showing score for the 5000 episodes to help visualize training progress and stability.

2.3 Population Method Algorithm:

Population-based optimization is a **gradient-free, black-box approach** to policy learning in RL. This method evaluates and evolves a set of perturbed policy parameters through repeated interaction with the environment. The fundamental idea is to explore the parameter space by sampling multiple perturbations of a policy, evaluating each based on cumulative reward, and selecting the best-performing parameter vector. This process is repeated for a predefined number of episodes, gradually improving the policy based on sampled performance.

<u>Algorithm Steps performed:</u>

1. Implemented parametric policy network by using simple neural network with 8 inputs, 128 hidden neurons, and 2 outputs using Tanh activation to bound the outputs between -1 and 1. Policy Optimization iteration starts with the below step.
2. A population of N = 20 candidate policies is generated by adding Gaussian Perturbation ϵ (noise = torch.randn_like(param) * std_dev σ = 0.02) to the current policy parameters.
3. For each candidate θ_i, the policy is averaged over 5 evaluations to reduce noise in reward scores, and the mean cumulative reward is recorded.
4. The candidate policy with the highest average reward is selected as the policy for the next episode, effectively updating $\theta \leftarrow \theta_i$_best. In summary, the perturbation mechanism here is generating N=20 perturbed policies (θi), evaluating each, and then selecting the best performing θi to become the new θ. This "survival of the fittest" approach, even with random perturbations, aims to gradually discover better performing policies.
5. Repeated steps 2-4 for 1,000 episodes, with rewards for each episode logged
6. The final policy parameters are saved for subsequent evaluation or deployment. The best and worst performing episodes is also tracked and logged.
7. Learning curve plotted showing score for the 1000 episodes to help visualize policy optimization

3. RESULTS – Plots and Observations:

The below two plots illustrates the learning curves of the RL agent trained using Zeroth-order optimization algorithm and Population Method Algorithm respectively on the

LunarLanderContinuous-v3 environment. The x-axis represents the Episode number, indicating the progression of training, while the y-axis represents the Return, which is the cumulative reward obtained by the agent in each episode. An episode is considered to be solved by an agent when a cumulative reward above 200 is obtained.

A. Zeroth-order optimization method

Fig a. Zeroth order with 5000 iterations

Observations:

- **Initial Exploration and Negative Returns (Episodes 0 - ~1600):** In the early episodes, the Return is consistently negative, ranging approx. from -100 to -50. This indicates that the RL agent's initial random policy is performing poorly, likely failing to land the lunar lander successfully and incurring large penalties. During this phase, the zeroth-order optimization algorithm is exploring the parameter space θ of the policy π. The perturbation mechanism is able to achieve small, incremental improvements. The relatively flat yet fluctuating curve suggests that the algorithm is making small, exploratory steps, and the environment's complexity combined with the initial random policy leads to consistent negative rewards.
- **Rapid Improvement Phase (Episodes ~ 1600 - 2500):** After around episode 1600, a noticeable upward trend begins, with the Return showing consistent and sharp incremental increases in Return ascending steeply from around -50 to over 250. This signifies a breakthrough in learning, where the agent rapidly identifies and refines highly effective landing strategies. The perturbation logic, although approximate, is able to guide the policy parameters towards more successful outcomes
- **Plateau and Convergence Phase (Episodes ~ 2500-5000):** In the latter half of the training, the Return largely stabilizes and plateaus, hovering around 250-270. This suggests that the agent's policy has converged to a stable and effective policy for the

LunarLanderContinuous-v3 environment given the algorithm's specific parameters. The minor fluctuations in this phase reflects the stochasticity of the environment and the exploration-exploitation trade-off.

B. Population method algorithm

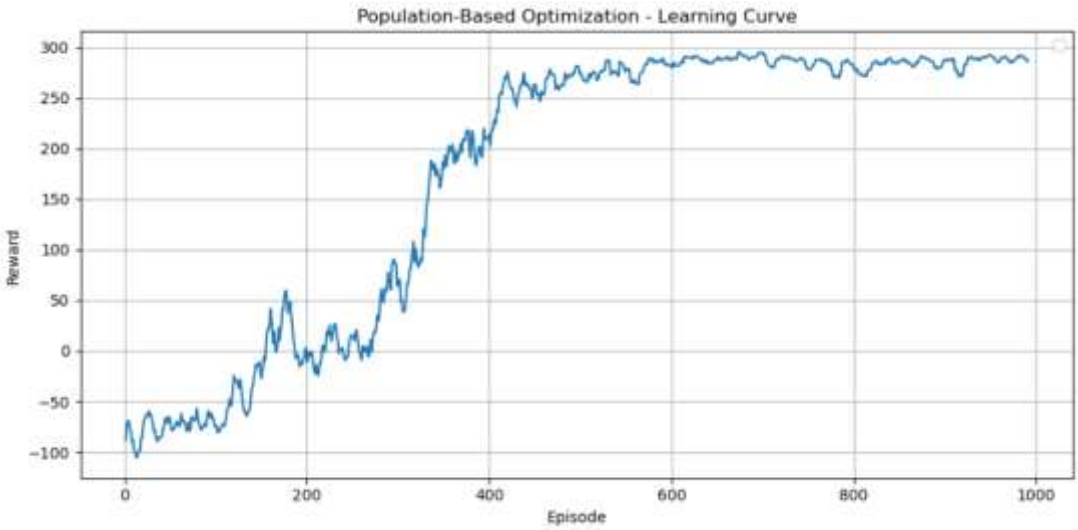


Fig b. Population method with 1000 iterations

Observations:

- **Initial Exploration and Negative Returns (Episodes 0 - ~150):** In the initial episodes of training, the Return is mostly negative, ranging from approx. -100 to 0. This indicates that the initial population of candidate policies is performing poorly, failing to land the lunar lander successfully and incurring large penalties. During this phase, the Population Method optimization algorithm is exploring the parameter space (θ) of the policy π. The fluctuating but generally upward trend within this negative range suggests that even in the initial exploration, the population method is gradually finding slightly better policies, although still far from optimal.
- **Period of Rapid Improvement (Episodes ~150 - 450):** In this phase, the RL agent's performance shows steep and consistent improvement, rapidly improving from negative returns, crossing into positive territory and towards high positive values, reaching peak score of approximately 280. This signifies a highly effective learning period where the Population-Based Optimization algorithm efficiently identifies and exploits better policy configurations. This phase confirms the effectiveness of the perturbation mechanism used in the algorithm leading the search towards more optimal regions of the parameter space.
- **Plateau and Convergence (Episodes ~450 - 1000):** In the latter half of the training, the Return largely stabilizes and plateaus, hovering around 280-290, with minor oscillations. This suggests that the RL agent's policy has largely converged to a near-optimal solution for the LunarLanderContinuous-v3 environment. The minor fluctuations within this plateau could be due to the stochastic nature of the environment or the continuous small perturbations that occasionally lead to slightly

different, but still highly performing, policy parameters. The stability at a high return value (approaching 300, which is typically considered the maximum for this environment) suggests that the algorithm has successfully found a robust and effective policy given the algorithm's parameters.

4. CONCLUSION:

This report presented an experimental comparison of two distinct gradient-free optimization algorithms – Zeroth-Order Optimization and Population-Based Optimization – applied to a Reinforcement Learning task within the LunarLanderContinuous-v3 environment. The objective was to evaluate their learning efficiency and final performance.

Based on the experimental results, we can conclude that **Population-Based Optimization outperforms Zeroth-Order Optimization** for the LunarLanderContinuous-v3 environment under the described experimental setups, due to the following key reasons:

1. **Accelerated Convergence**: Population-Based Optimization achieved a high level of performance significantly faster, reaching near-optimal returns within 450 episodes compared to Zeroth-Order's more prolonged learning over 3500 episodes.
2. **Enhanced Final Performance**: Population-Based Optimization consistently yielded slightly higher average returns (280-290) compared to Zeroth-Order's returns (250-270), indicating that Population-Based Optimization was able to discover a more effective and robust policy for successful lunar landings.
3. **Enhanced Stability at Convergence:** Both algorithms demonstrate stability at their respective plateaus. However, Population-Based Optimization reaches this stable, high-performance state much earlier in the training process and maintains it consistently over its training duration.

In conclusion, for this specific RL task and environment, Population-Based Optimization proved to be the more efficient and effective gradient-free method, offering considerably faster convergence and slightly higher performance compared to Zeroth-Order Optimization. These findings advocate for the strength of population-based approaches in scenarios where gradient information is unavailable or computationally expensive to obtain.

REFERENCES:

1. Richard S. Sutton and Andrew G. Barto c 2014, 2015. Reinforcement Learning: An Introduction. https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf

2. "Gymnasium Documentation." https://gymnasium.farama.org/environments/box2d/lunarlander.html